

DEM Velocity and Time Integration Summary

MechSys workspace note

2026-07-05

Scope

This note summarizes the velocity, contact-velocity, and time-integration approaches currently used in the DEM code paths under `lib/dem`. The main files are:

- `lib/dem/particle.h`: host particle velocity-Verlet primitives.
- `lib/dem/domain.h`: host and CUDA solve-loop orchestration.
- `lib/dem/dem.cuh`: CUDA kernels for force and velocity-Verlet updates.
- `lib/dem/interacton.h`: contact relative velocity, tangential history, and force models.

Particle State

Each particle carries translational and rotational state:

- Position: `x`, previous Verlet position `xb`.
- Linear velocity: `v`, half-step velocity `v_half`.
- Angular velocity: `w`, half-step angular velocity `w_half`.
- Orientation: quaternion `Q`.
- Net force/torque: `F`, `T`.
- Prescribed force/torque: `Ff`, `Tf`.
- Damping coefficients: `Gv` for linear velocity damping, `Gm` for angular velocity damping.
- Fixity flags: `vx`, `vy`, `vz`, `wx`, `wy`, `wz`.

The angular velocity is treated in the body frame for rotational integration. For contact kinematics it is rotated into the world frame using `Rotation(w,Q,t)`.

Contact Velocity

For a contact between particles 1 and 2, the code computes the contact points `x1c`, `x2c`, branch/contact normal `n`, and lever arms:

```
x1 = x1c - P1->x
x2 = x2c - P2->x
```

The world-frame angular velocities are obtained with:

```
Rotation(P1->w, P1->Q, t1)
Rotation(P2->w, P2->Q, t2)
```

The relative contact velocity is:

```
vrel = -((P2->v - P1->v) + cross(t2,x2) - cross(t1,x1))
```

or equivalently in the CUDA kernels:

```
vrel = (DPar[i1].v + cross(t1,x1)) - (DPar[i2].v + cross(t2,x2))
```

The tangential component is projected out of the normal direction:

```
vt = vrel - dot(n,vrel)*n
```

This `vt` drives tangential displacement history:

```
Fd += vt*dt
Fd -= dot(Fd,n)*n
```

The tangential elastic force is typically $Kt*Fd$. Normal and tangential dashpots use $Gn*dot(n,vrel)*n$ and $Gt*vt$.

CPU Velocity-Verlet Path

The default solver uses velocity Verlet when `UseVelocityVerlet` is true. The host implementation is split across `Particle` methods and `Domain::Solve`.

Initial Acceleration

Before the loop, contacts are built, forces are initialized from Ff/Tf , and contact forces are computed once. The solver then forms cached translational and rotational accelerations:

```
A = (F - Gv*m*v)/m
Wdot = I-1 * (T - Gm*I*w + gyroscopic terms)
```

Fixed velocity or angular-velocity components zero the corresponding force or torque components before acceleration is formed.

Step Sequence

For each time step:

1. Half-step velocities:

```
v_half = v + 0.5*dt*(F - Gv*m*v)/m
w_half = w + 0.5*dt*wdot(T,w)
```

2. Position and orientation updates:

```
x += dt*v_half
Q = normalize(Q * Exp(w_half*dt))
```

3. Force reset and contact force evaluation at the updated configuration:

```
v = v_half
w = w_half
F = Ff
T = Tf
CalcForce(dt,...)
```

Contact forces therefore use the updated positions and half-step velocities.

4. Full-step velocity completion:

```
v = v_half + 0.5*dt*(F_new - Gv*m*v_half)/m
w = w_half + 0.5*dt*wdot(T_new,w_half)
```

5. Maximum displacement is checked. If it exceeds `Alpha`, contact lists are rebuilt.

CUDA Velocity-Verlet Path

The CUDA path mirrors the CPU sequence but must cache accelerations explicitly. This is because `pReset` clears device forces before the next half-step, so the previous force is not available in `DPar[ic].F` at the moment `VerletStep1` runs.

Cached Arrays

The device stores:

- **pA**: cached translational acceleration for the next half-step.
- **pWdot**: cached angular acceleration for the next half-step.

These arrays must contain the same damped accelerations used by the previous full-step update. If they contain undamped or stale values, the next GPU half-step diverges from the CPU path.

Kernel Sequence

Each CUDA velocity-Verlet step is:

1. Reset forces and contact outputs:

```
pReset(...)
```

2. Half-step velocity and position update:

```
pVerletStep1(..., pA, ...)
```

This computes **v_half**, advances **x**, moves vertices, and stores the half-step velocity in **DPar[ic].v**.

3. Half-step angular velocity and orientation update:

```
pOrientationUpdate(..., pWdot, ...)
```

This computes **w_half**, updates **Q** by quaternion exponential map, rotates vertices, and stores **w_half** in **DPar[ic].w**.

4. Contact force kernels:

```
pForceVV(...)  
pForceEE(...)  
pForceVF(...)  
pForceFV(...)
```

These use the updated positions and half-step velocities, matching the CPU force-evaluation point.

5. Full-step velocity and acceleration-cache update:

```
pFinalizeVelocity(..., pA, ...)  
pFinalizeRotation(..., pWdot, ...)
```

These complete **v** and **w**, then store the same damped accelerations into **pA** and **pWdot** for the next step.

6. Wrapping/contact-list update logic:

MaxD computes displacement. If contact lists are rebuilt, device state is downloaded, contacts are rebuilt on host, device contact data is uploaded, and **RecomputeAccelerations** refreshes **pA/pWdot** from the current force state.

Quaternion Convention

Host quaternions use scalar-first notation:

$Q = (q_0, q_1, q_2, q_3)$

CUDA **real4** now uses the same scalar-first order:

$Q = (w, x, y, z)$

Therefore the CUDA identity quaternion is:

```
make_real4(1, 0, 0, 0)
```

and the exponential map for angular velocity **omega** is:

```

dq.x    = cos(theta/2)
dq.yzw  = sin(theta/2) * axis
theta   = |omega|*dt

```

The quaternion product must also return (w, x, y, z) in that order. Keeping CPU and GPU storage aligned avoids the old upload/download remapping step and reduces the risk of rotating angular velocities with the wrong scalar/vector slots.

Legacy Position-Verlet / Leapfrog Path

When `UseVelocityVerlet` is false, the solver uses the older formulation:

1. Reset forces and torques.
2. Compute forces from current positions and velocities.
3. Translate with position Verlet:

```

xa = 2*x - xb + dt^2*F/m
v  = 0.5*(xa - xb)/dt
xb = x
x  = xa

```

4. Rotate with the older rotational update.
5. Check displacement and rebuild contacts if needed.

This path is less aligned with velocity-dependent forces because forces are computed before the position update rather than at the velocity-Verlet midpoint.

Contact Model Notes

For general feature contacts and sphere contacts, the contact model combines:

- Elastic normal force: $F_n = K_n \cdot \delta \cdot n$.
- Normal dashpot: $G_n \cdot \text{dot}(n, v_{rel}) \cdot n$.
- Tangential displacement history: $F_d += v_t \cdot dt$.
- Tangential elastic force: $K_t \cdot F_d$.
- Tangential dashpot: $G_t \cdot v_t$.
- Coulomb limiting through μ .

Sphere contacts expose options controlling how the Coulomb cap treats tangential elastic and dashpot components. The important integration point is unchanged: the force model should see the same contact velocity state on CPU and GPU.

Practical Checklist

For CPU/GPU trajectory agreement:

1. `pA` and `pWdot` must be initialized from the same damped force/torque state as the CPU half-step.
2. `FinalizeVelocity` and `FinalizeRotation` must store the damped acceleration they actually used.
3. `RecomputeAccelerations` after contact-list rebuilds must not leave stale acceleration caches.
4. CUDA quaternion updates must respect scalar-first (w, x, y, z) storage.
5. Contact kernels must evaluate forces after position/orientation updates, with `DPar.v` and `DPar.w` holding half-step values.
6. Fixed velocity/angular-velocity flags should be applied consistently before forming accelerations.